

Practical Part - Advanced: Collaborative Git with GitHub

This section will guide you through collaborative Git workflows using GitHub. We'll work in small teams to experience how Git facilitates shared development, code reviews, and conflict resolution in a real-world scenario.

Project Setup: A Simple Web Page

We'll be working on a basic web page with a simple counter. Here's the initial structure:

- `index.html`: Contains the HTML structure for a counter display and buttons.
- `script.js`: Contains JavaScript to increment/decrement the counter.
- `style.css`: Contains basic styling for the page.

1. Create Your GitHub Repository

Initially, only one of the team members will create the GitHub repository. This person will be the "owner" of the repository, and the others will be collaborators.

First, we need to create a repository on GitHub that will host our project. This repository will be the central hub for our collaborative work.

- Go to [GitHub](#) and create a new repository, name it whatever you want.
- **Do not** initialize it with a README, `.gitignore`, or license. We'll add these later.
- Copy the repository's HTTPS or SSH URL. You'll need this to clone the repository.

Instructions:

1. Log in to your GitHub account.
2. Click the "+" button in the top right corner and select "New repository".
3. Enter the repository name.
4. Leave the "Initialize this repository with" options unchecked.
5. Click "Create repository".
6. Copy the HTTPS or SSH URL of your new repository.

2. Clone the Repository

Now, we'll clone the empty repository to our local machine.

- On your local machine, use `git clone <your-repository-url>` to clone the empty repository.
- `cd {repo_name}` to enter the directory.

1. Create the Initial Files

We'll create the initial HTML, JavaScript, and CSS files that form the base of our counter application.

1. Create the `index.html`, `script.js`, and `style.css` files in the project directory using your text editor or IDE.

2. Copy and paste the provided code into the respective files.

index.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Counter</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div id="counter">0</div>
  <button id="increment">+</button>
  <button id="decrement">-</button>
  <script src="script.js"></script>
</body>
</html>
```

script.js:

```
const counterElement = document.getElementById('counter');
const incrementButton = document.getElementById('increment');
const decrementButton = document.getElementById('decrement');

let count = 0;

incrementButton.addEventListener('click', () => {
  count++;
  counterElement.textContent = count;
});

decrementButton.addEventListener('click', () => {
  count--;
  counterElement.textContent = count;
});
```

style.css:

```
#counter {
  font-size: 2em;
  text-align: center;
  margin: 20px;
}
button {
  font-size: 1.5em;
  padding: 10px;
}
```

3. Run `git add .` to stage all changes.
4. Run `git commit -m "Initial commit"` to commit the changes.
5. Run `git push origin master` to push the changes to the remote repository.

4. Invite Collaborators

Now, we'll simulate a collaborative workflow by having each team member work on a separate feature or change in their own branch.

- Divide into groups of 2-3 people.
- Each person clones the repository to their local machine using `git clone <your-repository-url>`.
- Each person creates their own branch: `git checkout -b feature/{yourname}`.
- Each person implements a feature or changes the code in their branch. Examples:
 - Add a reset button.
 - Add a display of the current time the counter was changed.
 - Change the style of the counter.
 - Add a counter limit.

Feel free to use chatGPT if you don't know the specifics on how to implement a feature.

Make sure to commit and add a descriptive commit message.

5. Push Your Branch

Once you've made your changes, push your branch to the remote repository.

1. Run `git push origin feature/{yourname}` to push your branch to the remote repository.
2. Go to the GitHub repository page and verify that your branch appears in the "branches" section.

6. Create a Pull Request

Now that each team member has pushed their changes to their own branch, it's time to create a pull request (PR) to merge those changes into the main branch.

1. Go to the GitHub repository page.
2. Click on the "Pull requests" tab.
3. Click on the "New pull request" button.
4. Select your branch from the "compare" dropdown.
5. Click on "Create pull request".
6. Add a title and description for your pull request.
7. Click on "Create pull request".

7. Code Review

Code review is a crucial part of collaborative development. It allows team members to provide feedback and ensure code quality.

Each team member reviews the pull requests of others. Provide feedback and comments.

1. Go to the "Pull requests" tab in your GitHub repository.
2. Click on the pull request you want to review.
3. Review the changes made in the pull request.
4. Add comments or suggestions for improvement.

8. Resolve comments from the code review and merge the pull request

1. If there are comments or suggestions from the code review, address them in your branch.
2. Merge the pull request into the main branch.
3. Click on the "Merge pull request" button.
4. Confirm the merge.
5. Delete the branch if it's no longer needed.

Merge conflicts

Play a bit around with your code, attempt to create more changes, more commits, create some merge conflicts and try to resolve them.